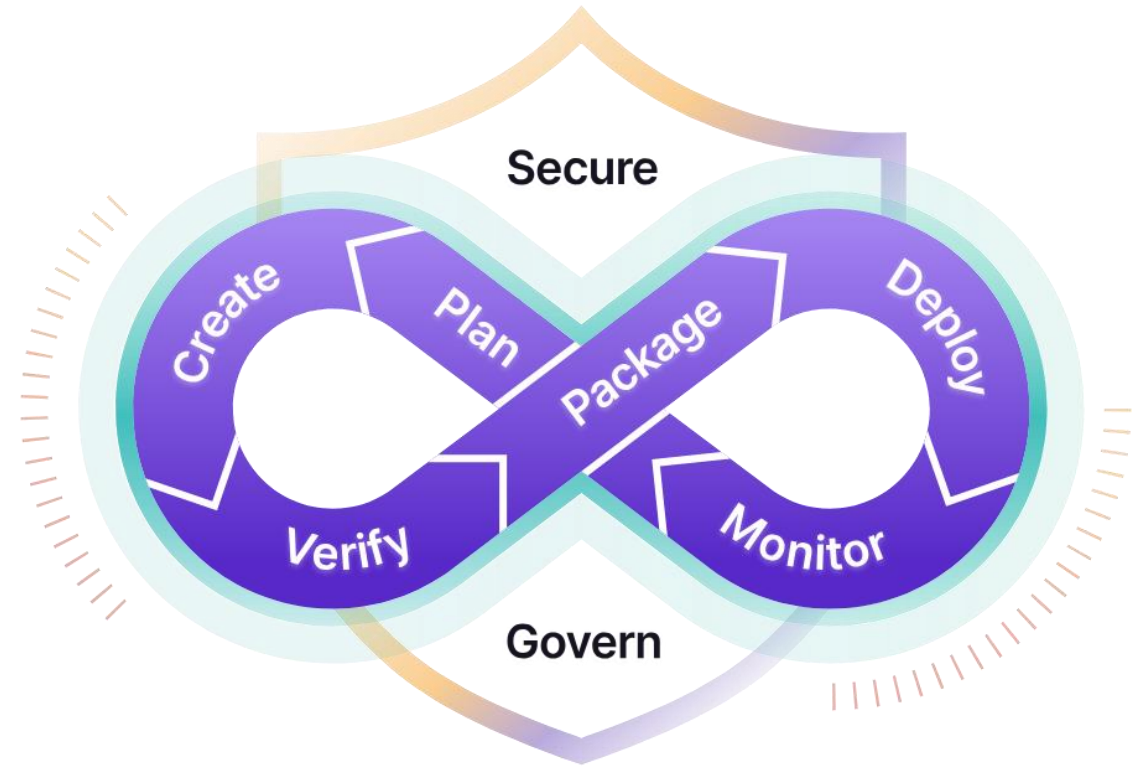




DevOps & CI/CD

What is Devops

- DevOps combines development (Dev) and operations (Ops) to increase the efficiency, speed, and security of software development and delivery compared to traditional processes. A more nimble software development lifecycle results in a competitive advantage for businesses and their customers.
- Stemming from an Agile approach to software development, a DevOps process expands on the cross-functional approach of building and shipping applications in a faster and more iterative manner.



Core DevOps principles

1. Automation of the software development lifecycle

This includes automating testing, builds, releases, the provisioning of development environments, and other manual tasks that can slow down or introduce human error into the software delivery process.

2. Collaboration and communication

A good DevOps team has automation, but a great DevOps team also has effective collaboration and communication

3. Continuous improvement and minimization of waste

From automating repetitive tasks to watching performance metrics for ways to reduce release times or mean-time-to-recovery, high performing DevOps teams are regularly looking for areas that could be improved.

4. Hyperfocus on user needs with short feedback loops.

Through automation, improved communication and collaboration, and continuous improvement, DevOps teams can take a moment and focus on what real users really want, and how to give it to them.

The DevOps lifecycle and how DevOps works

- **Plan:** Organize the work that needs to be done, prioritize it, and track its completion.
- **Create:** Write, design, develop and securely manage code and project data with your team.
- **Verify:** Ensure that your code works correctly and adheres to your quality standards — ideally with automated testing.
- **Package:** Package your applications and dependencies, manage containers, and build artifacts.
- **Secure:** Check for vulnerabilities through static and dynamic tests, fuzz testing, and dependency scanning.
- **Release:** Deploy the software to end users.
- **Configure:** Manage and configure the infrastructure required to support your applications.
- **Monitor:** Track performance metrics and errors to help reduce the severity and frequency of incidents.
- **Govern:** Manage security vulnerabilities, policies, and compliance across your organization.

DevOps tools, concepts and fundamentals

- **Version control:** The fundamental practice of tracking and managing every change made to source code and other files. Version control is closely related to source code management.
- **Agile:** Agile development means taking iterative, incremental, and lean approaches to streamline and accelerate the delivery of projects.
- **Continuous Integration (CI):** The practice of regularly integrating all code changes into the main branch, automatically testing each change, and automatically kicking off a build.
- **Continuous Delivery (CD):** Continuous delivery works in conjunction with continuous integration to automate the infrastructure provisioning and application release process. They are commonly referred to together as CI/CD.
- **Shift left:** A term for shifting security and testing much earlier in the development process. Doing this can help speed up development while simultaneously improving code quality.

How does DevSecOps relate to DevOps?

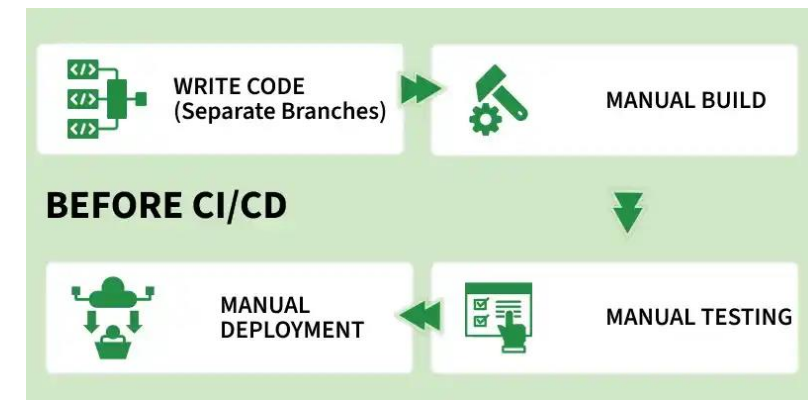
- DevSecOps extends DevOps by embedding security checks and compliance into every stage of the development lifecycle.
- Security has become an integral part of the software development lifecycle, with much of the security shifting left in the development process. DevSecOps ensures that DevOps teams understand the security and compliance requirements from the very beginning of application creation and can properly protect the integrity of the software.
- By integrating security seamlessly into DevOps workflows, organizations gain the visibility and control necessary to meet complex security demands, including vulnerability reporting and auditing. Security teams can ensure that policies are being enforced throughout development and deployment, including critical testing phases.

What is CI/CD?

- CI/CD stands for Continuous Integration and Continuous Delivery/Deployment, a practice that automates the process of building, testing, and releasing software.
- It forms the backbone of modern DevOps by connecting development and operations to deliver applications faster and more reliably.
 - It automates code integration, testing, and deployment workflows.
 - Reduces manual effort while improving software quality.
 - Enables faster, more consistent, and reliable releases.

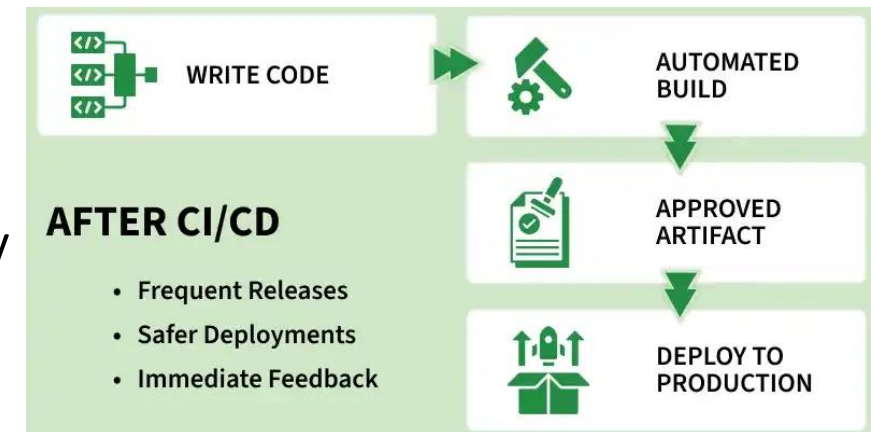
Before CI/CD

- In the past, software delivery was slow and full of manual work. Developers used to create separate feature branches and worked on them for weeks or even months.
- All branches were merged at the end, which often caused big conflicts and broken builds. Testing and building the code were also manual, usually done at the final stage, so bugs were discovered very late and were costly to fix.
- Deployment was a long and risky process, taking days or sometimes weeks, because everything was released together in one big update.
- Teams were also divided: developers wrote code, testers tested it, and operations managed servers.



After CI/CD

- With CI/CD, the whole process became faster, automated, and more reliable. Developers now commit their code frequently to a shared main branch, which prevents conflicts and ensures smooth integration.
- Automated pipelines run tests, builds, and checks as soon as code is pushed, so errors are caught early. Continuous Delivery makes sure that tested code is automatically packaged and ready for deployment.
- While Continuous Deployment takes it a step further by directly releasing to production without manual steps. This means smaller, frequent updates instead of one big risky release.
- Tools like Git, Jenkins, Docker, and Kubernetes automate everything from building to deployment, making the workflow transparent and collaborative. Bugs are fixed quickly, rollbacks are easier, and overall software delivery happens in hours instead of days.



The Three Pillars of CI/CD

1. Continuous Integration (CI): The Goal: Prevent "Integration Hell."

The Process: Developers merge their changes back to the main branch as often as possible (daily).

The Automation: Upon every commit, an automated system builds the code and runs unit tests. If a test fails, the build is rejected, and the developer is notified immediately.

2. Continuous Delivery (CD): The Goal: Be ready to release at any moment.

The Process: After CI passes, the code is automatically deployed to a staging/testing environment. Extensive integration and load tests are run.

The Release: The codebase is always in a deployable state, but the push to production is a manual decision (e.g., a "Deploy" button click by a manager).

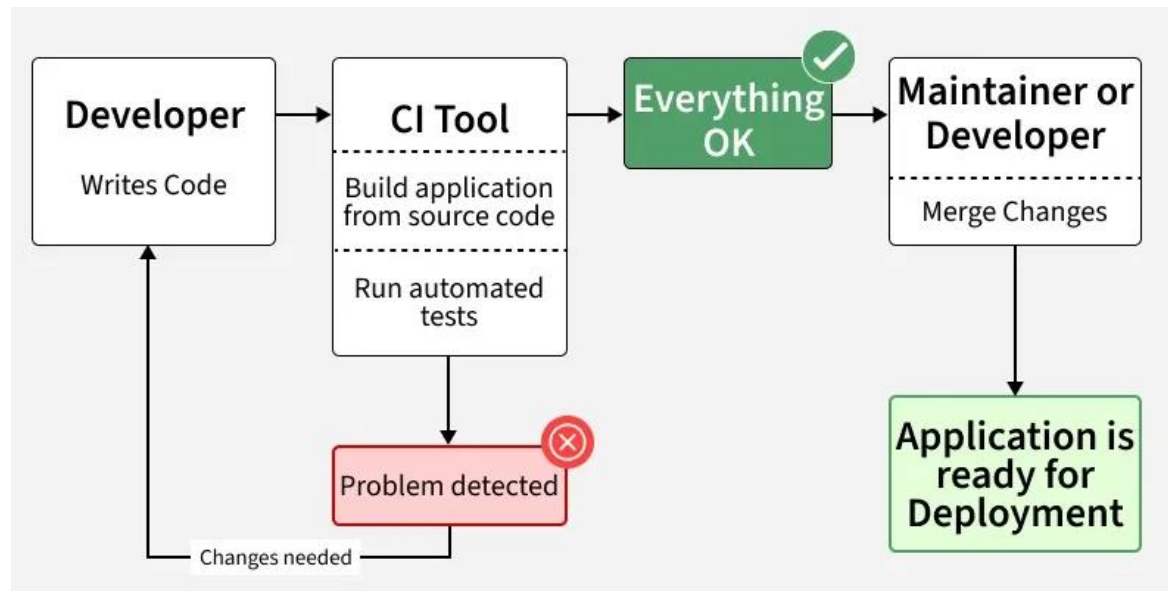
3. Continuous Deployment (CD): The Goal: Eliminate human intervention.

The Process: Same as Continuous Delivery, but if all automated tests pass in staging, the code is automatically deployed to production.

The Requirement: Requires extremely robust, trustworthy automated testing suites

CI Workflow

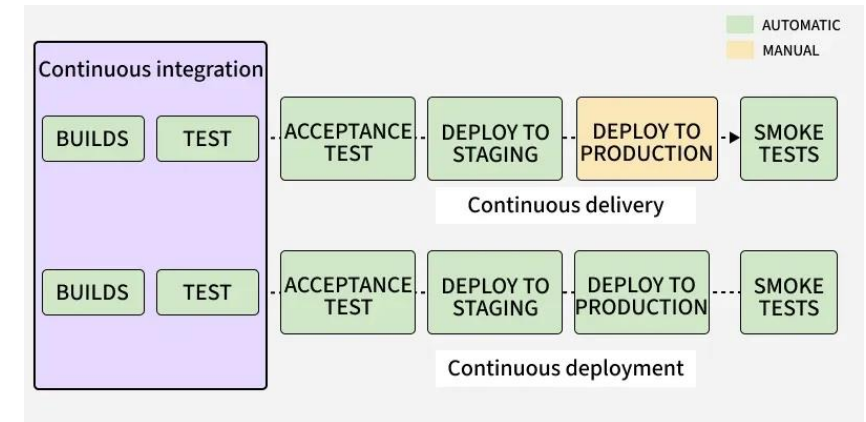
- When a developer saves (commits) their code to a version control system like Git, the CI pipeline starts. It takes the new changes, builds the project, and runs tests automatically. After that, the server tells the developer if adding the new code to the existing code worked or failed.
- This helps find and fix bugs faster, saves developers from doing the same tasks again and again, and allows the team to release updates to customers more often. Studies show that using CI can save developers about 25–30% of their time.



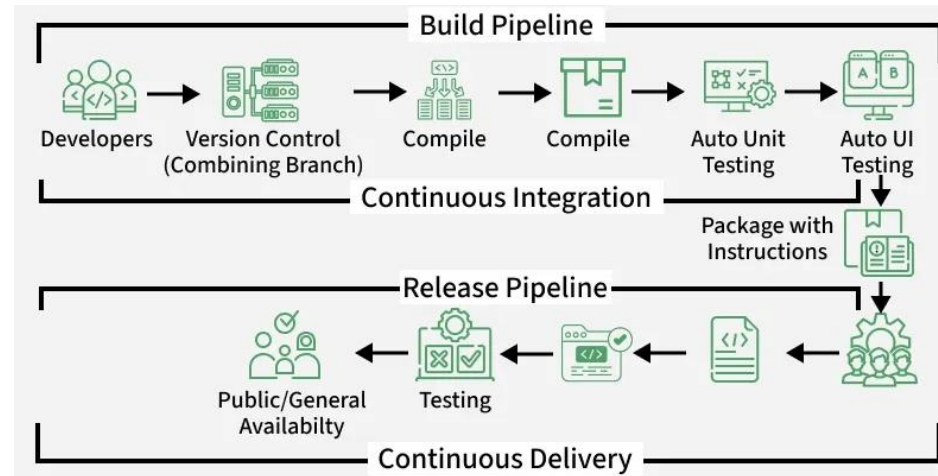
CI and CD Workflow

We have seen how Continuous Integration automates the process of building, testing, and packaging the source code as soon as it is committed to the code repository by the developers. After the Continuous Integration (CI) step, the code is deployed to a staging environment.

- In the staging environment, it undergoes further automated testing such as: Acceptance Testing & Regression Testing
- Once tests are successful, the code is deployed to the production environment for the final release.
- If the production deployment is manual, the process is called Continuous Delivery.
- If the production deployment is automated, the process is called Continuous Deployment.



What is CI/CD Pipeline?

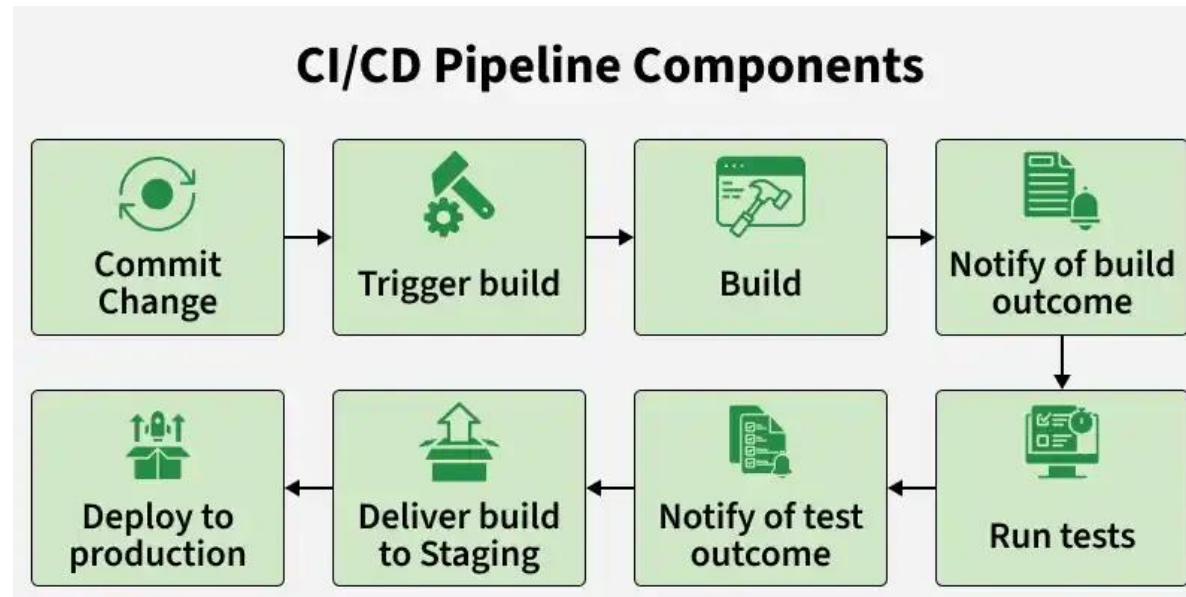


Many people think CI/CD and CI/CD Pipeline are the same, but they are a bit different.

- CI/CD (Continuous Integration and Continuous Delivery): This is a way of working where developers can make many code changes and deliver updates quickly.
- CI/CD Pipeline: This is the actual step-by-step process that takes the code and automatically moves it from development to live production.

In simple words, a CI/CD pipeline is a series of automatic steps that help developers build, test, and deliver code safely and quickly.

Components of a CI/CD Pipeline



1. **Commit Change:** Developers make code changes and commit these changes to a version control system (e.g., Git). This step initiates the CI/CD pipeline. Committing code changes ensures that they are tracked and versioned properly.
2. **Trigger Build:** The version control system detects the new commit and triggers the build process automatically. Automated triggering of the build process ensures that new changes are continuously integrated and tested.

Components of a CI/CD Pipeline

3. Build

The codebase is compiled and built into a deployable artifact, such as a binary executable or a Docker image. Common tools include Maven, Gradle, Ant for Java projects, and Docker for containerized applications. The build step verifies that the code compiles correctly and that all dependencies are resolved.

4. Notify of Build Outcome

The CI/CD system notifies the team of the build results, whether it passed or failed. Immediate feedback on the build status helps developers quickly identify and resolve any build issues. Notification systems can include emails, chat integrations (like Slack or Microsoft Teams), or dashboards in tools like Jenkins or GitLab CI.

5. Run Tests

Automated tests are executed on the build artifact. These can include unit tests, integration tests, end-to-end tests, and more. Testing frameworks like JUnit, Selenium, TestNG, pytest, etc. Running tests ensures that the new code does not introduce any bugs or regressions and that it meets the required quality standards.

Components of a CI/CD Pipeline

6. Notify of Test Outcome

The results of the test suite are reported back to the development team. Quick feedback on test outcomes allows developers to address any failures promptly. Similar to build notifications, results can be sent via email, chat, or dashboards.

7. Deliver Build to Staging

If tests pass, the build artifact is deployed to a staging environment. The staging environment simulates the production environment, allowing for final validation before production deployment. Deployment tools like Ansible, Chef, Puppet, Kubernetes, or cloud-specific services like AWS CodeDeploy.

8. Deploy to Production

After successful validation in staging, the build is automatically or manually promoted to the production environment. This step makes the new features and fixes available to end-users. Deployment strategies might include blue-green deployment, canary releases, or rolling updates to minimize downtime and risk.